

PERS — Incident Management & RRT Dispatch: Architecture & Documentation

Oct 1, 2026 · @Bimad

Overview & Delivery Scope

PERS is one Next.js application, one Supabase project and one Mapbox account that together give a control room live visibility of towers and Rapid Response Teams (RRTs), automatic dispatch of the nearest team, and a full audit trail from alarm to resolution report. Nothing is built yet; this document is the architecture and the plan you approve before I generate any code.

The product is delivered as three surfaces inside a single codebase and a single Vercel deployment:

Surface	URL path	Users	Purpose
Control Room Dashboard	/dashboard , /incidents , /towers , /teams , /reports	Operators, Administrators	Live map, stats, trigger incidents, monitor teams, reports
Admin Panel	/admin/*	Super Admin, Administrator	Tower and team CRUD, Excel import, question builder, users, settings
RRT Mobile PWA	/rrt/*	RRT team members	Installable Android app: alerts, accept/reject, navigate, live GPS, resolve form, photos

In scope (everything in your brief): 14-table PostgreSQL schema with RLS, views and triggers; realtime GPS, assignment, acceptance, arrival and resolution; Haversine dispatch with 30-second auto-escalation; configurable resolution forms with photo upload; daily, weekly, monthly and custom reports as PDF, Excel and CSV; seed data of 100 towers, 10 teams and 50 historical incidents; Docker support; deployment guide; test plan.

Added by me because the demo needs it: a Demo Controller that moves simulated teams along real roads and auto-accepts alerts, so a customer sees ten live teams while you hold one real phone. Section 2 explains why this matters.

Simplified for the demo, hardened for production later: background GPS when the phone is locked, SMS fallback, road-distance (instead of straight-line) ranking, multi-tenant separation per telecom operator, and offline-first sync. Each is listed with its upgrade path in the Risks section.

Decisions & Assumptions

Ten decisions below depart from, tighten or add to your brief; each one prevents a failure that would otherwise appear during the live demo. Tell me which you disagree with before the build starts.

#	Topic	Decision	Why it matters
1	30-second escalation	Runs inside PostgreSQL: each offer stores an <code>expires_at</code> , and a <code>pg_cron</code> job sweeps expired offers every 5 seconds and offers the incident to the next team	A timer in the phone or browser dies when the app closes or the screen locks. Database timing is the only version that is always correct. <code>pg_cron</code> supports 1 to 59 second schedules.
2	Arrival detection	Also in the database: a trigger on every GPS insert measures distance to the tower and flips the incident to REACHED under 50 m	One authoritative rule, identical for real phones and simulated teams.
3	GPS when phone is locked	The PWA sends GPS every 10 seconds only while the app is open and the screen is kept awake (Screen Wake Lock)	Android Chrome pauses web pages when the screen locks. Fine for a demo with the phone on a dashboard mount; production needs a native wrapper (Section 20).

#	Topic	Decision	Why it matters
4	Alerts to a closed app	Web Push (service worker) plus an in-app popup with siren sound and vibration	Push wakes an installed PWA on Android. The in-app 30-second countdown popup is driven by the offer's <code>expires_at</code> , so it is always in sync.
5	HEIC photos	Converted to JPEG on the phone before upload (original optionally kept)	Browsers and PDF libraries cannot display HEIC; converting once makes reports and the dashboard work.
6	Ranking distance	Haversine straight-line distance ranks the teams, as you specified. Mapbox Directions then gives road distance and ETA once a team accepts	Haversine is instant and free; road routing for every candidate would cost API calls and add latency.
7	Demo Controller	An admin page that drives simulated teams along real roads and can auto-accept alerts after a delay you set	A customer wants to see 10 live cars. You will hold one real phone; the other nine are simulated and flagged as such in the database.
8	One codebase	The mobile PWA is the <code>/rrt</code> section of the same Next.js app, with its own manifest and scope	One deploy, one auth system, one set of types. Installable on Android from Chrome with no Play Store.
9	Supabase keys	Browser uses the publishable key (<code>sb_publishable_...</code>); only server code uses the secret key (<code>sb_secret_...</code>). Older projects label these <code>anon</code> and <code>service_role</code>	The secret key bypasses all Row Level Security. It must never reach the browser or GitHub.
10	Plans for the demo	See below	Free plans have hard rules that can break a customer demo.

Plan check, as of 1 October 2026. Vercel's Hobby plan is limited to non-commercial, personal use; a demo for a telecom customer is better run on Pro at \$20 per developer seat per month. Supabase Free pauses a project after one week of inactivity and allows 200 concurrent Realtime connections; Pro is \$25 per month, never pauses and allows 500 connections. Mapbox's free quotas (50,000 web map loads, 100,000 Directions requests and 50,000 Static Images requests per month) are far above what a demo uses, so Mapbox stays free. Sources are listed in Section 21.